statnnet: Statistical Selection and Interpretation for Neural Networks Fitted using nnet

by Andrew McInerney and Kevin Burke

Abstract The **nnet** package provides a long-established implementation of single-hidden-layer feedforward neural networks in R, but its standard workflow is primarily concerned with model fitting and prediction. The **statnnet** package extends supported **nnet** models with a statistical workflow for small, parsimonious networks. It provides information-criterion selection of input terms and hidden-layer size, approximate covariance estimation, Wald-type summaries for individual and grouped network weights, finite-difference covariate-effect summaries, partial covariate-effect plots and contrasts with uncertainty estimates, and diagnostics for model selection, optimisation, and covariance estimation. These quantities are integrated through familiar R interfaces including `summary()`, `anova()`, `coef()`, `vcov()`, `predict()`, and `plot()`. This paper describes the statistical quantities implemented by **statnnet**, its software design and limitations, and demonstrates the resulting workflow through an applied regression example.

1 Introduction

Feedforward neural networks (FNNs) are flexible non-linear models that are widely used for prediction and function approximation (LeCun et al., 2015). Although commonly associated with machine learning and artificial intelligence, they can also be viewed as non-linear regression models in which a vector of covariates is mapped to a conditional mean through weighted linear combinations and non-linear activation functions. This statistical perspective is particularly relevant for tabular data, where relatively small neural networks remain common in applied work. In a systematic review of empirical comparisons between neural networks and generalised additive models (GAMs), Doohan et al. (2026) identified 143 papers analysing 430 real-world tabular datasets. Among the datasets for which network depth was reported, 70% used a single hidden layer and a further 22% used two hidden layers, indicating that shallow architectures continue to account for a substantial proportion of applied neural-network analyses. The review found no consistent predictive superiority of either neural networks or GAMs across the most frequently reported performance measures, while noting that GAMs retain an important advantage in interpretability.

These findings motivate attention not only to the predictive performance of shallow neural networks, but also to the statistical workflow available for analysing them. Flexible statistical models such as GAMs are accompanied by established tools for examining model terms, covariate effects, and uncertainty. Neural-network implementations, by contrast, have traditionally placed greater emphasis on fitting and prediction. This distinction is particularly relevant when relatively parsimonious networks are used as flexible non-linear regression models rather than as components of large-scale deep-learning systems. In such settings, prediction can be complemented by statistical summaries that help describe which covariates contribute to the fitted model, how the fitted response changes with those covariates, and whether the numerical properties of the fitted model support uncertainty-based interpretation.

In R, the **nnet** package (Venables and Ripley, 2002) provides a long-established implementation of single-hidden-layer FNNs, with optional skip-layer connections and weight decay. An object returned by `nnet()` contains fitted weights, fitted values, residuals, an objective value, and convergence information, and can also contain a Hessian matrix. Although **nnet** provides established functionality for model fitting and prediction, it does not directly provide covariate-level statistical summaries, grouped tests of model terms, covariate-effect estimates with uncertainty, or diagnostics designed to assess the suitability of covariance-based interpretation. Obtaining such quantities typically requires users to extract and reorganise network weights, reconstruct model matrices, calculate approximate covariance matrices, or combine several separate interpretation tools. This is the practical gap addressed by **statnnet**.

A wide range of model-agnostic methods is available for explaining fitted prediction functions, including Shapley-value approaches (Strumbelj and Kononenko, 2010), locally interpretable model explanations (LIME) (Ribeiro et al., 2016), partial dependence, and related methods. These approaches are valuable because they can be applied across many classes of predictive models, but their objective differs from treating a fitted FNN explicitly as a statistical model. For a suitably small FNN, the model parameterisation, fitting objective, and local curvature can themselves be informative: they permit

parameter covariance to be approximated, groups of network parameters associated with an original covariate to be considered jointly, and uncertainty to be propagated to function-scale summaries. Model-agnostic explanation and model-specific statistical interpretation are therefore complementary rather than competing approaches.

The **statnnet** package is designed to provide the latter for a deliberately defined class of models fitted using **nnet**. Rather than introducing another neural-network optimiser or attempting to provide a common interface across deep-learning frameworks, **statnnet** builds directly on an established R fitting engine whose parameter ordering, activation functions, optimisation objective, convergence information, and Hessian calculations can be handled explicitly. The restriction to parsimonious single-hidden-layer networks is therefore a design choice rather than an attempt to provide inference for arbitrary neural-network architectures. Such networks remain common in tabular applications, and their relatively small parameterisations make regression-style statistical analysis practically meaningful. The objective of **statnnet** is not to turn arbitrary neural networks into classical regression models, but to provide a statistically informed regression-style workflow for the shallow networks for which such an analysis is appropriate.

The package integrates several components of this workflow. The `select_nnet()` function provides information-criterion selection of input terms and hidden-layer size for the supported Gaussian setting. A selected or independently fitted `nnet` object can then be augmented by `statnnet()`, which validates the fitted model and provides approximate covariance estimation, Wald-type summaries for individual parameters and groups of input weights, covariate-level summaries for multi-column factors, finite-difference covariate-effect summaries, partial covariate-effect (PCE) plots and contrasts with uncertainty, prediction, network visualisation, and diagnostic information. These quantities are exposed through familiar R interfaces including `summary()`, `anova()`, `coef()`, `vcov()`, `plot()`, and `predict()`. An important part of this workflow is diagnostic rather than inferential: **statnnet** checks convergence, architecture, and the numerical behaviour of the information matrix before covariance-based summaries are reported, and distinguishes numerical computability from evidence that the associated reference distributions are well calibrated.

The statistical methodology implemented in the package was developed separately. McInerney and Burke (2025) introduced the information-criterion procedure for selecting input terms and hidden-layer size, while McInerney and Burke (2026) developed the Wald-type summaries, covariance approximation, and partial covariate-effect methodology. The contribution of the present paper is therefore principally software and workflow: it brings these methods together in an R implementation, extends them to a coherent model-object interface, handles formula terms and categorical covariates at their original covariate level, provides diagnostic safeguards, and makes supported `nnet` models accessible through an analysis workflow familiar from conventional statistical modelling. Existing R packages provide complementary tools for neural-network visualisation, sensitivity analysis, and model-agnostic interpretation; Section 2.3 considers these packages in detail and clarifies the distinct role of **statnnet**.

The paper is organised as follows. Section 2.2 introduces the FNN parameterisation, selection criterion, and statistical quantities used by **statnnet**. Section 2.3 positions the package relative to existing neural-network fitting and model-interpretability packages. Section 2.4 demonstrates the package workflow using an applied regression example. Section 2.5 summarises the supported model objects and S3 methods. Section 2.6 discusses practical limitations and appropriate interpretation of the resulting summaries. Section 2.7 concludes.

2 Statistical background

This section summarises the statistical quantities computed by **statnnet**. A fuller development of the methodology, including the derivation of the Wald-type summaries, simulation evidence, and practical recommendations, is given in McInerney and Burke (2026). Here, we introduce only the notation and definitions required to understand the package output.

The package is designed for supported single-hidden-layer models fitted using **nnet** that can be viewed as non-linear regression models. Let y_i denote the response for observation $i = 1, \ldots, n$, and let $x_i = (x_{i0}, x_{i1}, \ldots, x_{ip})^T$ denote the corresponding vector of covariates, where $x_{i0} \equiv 1$ is an intercept term. For a single-hidden-layer FNN with q hidden nodes, the conditional mean can be written as

$$\mathbb{E}(y_i \mid x_i) = \mathrm{NN}(x_i, \theta),$$

where

$$\mathrm{NN}(x_i, \theta) = \phi_o \left[\gamma_0 + \sum_{k=1}^{q} \gamma_k \phi_h \left(\sum_{j=0}^{p} \omega_{jk} x_{ij} \right) \right].$$

Here, ω_{jk} is the weight connecting input j to hidden node k, γ_k is the weight connecting hidden node k to the output node, γ_0 is the output intercept, and θ denotes the full vector of model parameters. The functions $\phi_h(\cdot)$ and $\phi_o(\cdot)$ are the hidden-layer and output-layer activation functions, respectively. For continuous responses, $\phi_o(\cdot)$ is typically the identity function; for binary responses, it is typically the logistic function.

The statistical calculations in **statnnet** use the fitting criteria implemented by **nnet** directly. Let $\delta \geq 0$ denote the scalar `decay` argument and let K be the number of fitted weights, including the hidden- and output-layer intercepts. For the supported linear-output regression model, `nnet()` minimises

$$Q_{\delta,G}(\theta) = \sum_{i=1}^{n} \{y_i - \mathrm{NN}(x_i, \theta)\}^2 + \delta\theta^T\theta.$$

For a binary factor response, `nnet()` uses a single logistic output with `entropy = TRUE` and minimises

$$Q_{\delta,B}(\theta) = -\sum_{i=1}^{n} [y_i \log\{\mathrm{NN}(x_i, \theta)\} + (1 - y_i) \log\{1 - \mathrm{NN}(x_i, \theta)\}] + \delta\theta^T\theta.$$

Thus, for the scalar `decay` argument considered here, **nnet** applies weight decay to all K fitted weights rather than excluding intercept weights. The component `value` is the relevant data-fitting criterion plus this weight-decay term.

Write $H_\delta = \nabla^2 Q_\delta(\hat{\theta})$ for the Hessian returned when `Hess = TRUE`. On the native **nnet** scale, the unpenalised curvature is

$$H_0 = H_\delta - 2\delta I_K,$$

where I_K is the $K \times K$ identity matrix. For binary entropy fitting, the corresponding likelihood-scale matrices are $J_0 = H_0$ and $J_\delta = H_\delta$. For Gaussian regression, the residual variance is estimated as $\hat{\sigma}^2 = n^{-1} \sum_{i=1}^{n} \{y_i - \mathrm{NN}(x_i, \hat{\theta})\}^2$, and the residual-sum-of-squares criterion is converted to the negative-log-likelihood scale using

$$J_0 = \frac{H_0}{2\hat{\sigma}^2}, \qquad J_\delta = \frac{H_\delta}{2\hat{\sigma}^2}.$$

The covariance matrix for the penalised estimate is then approximated by the sandwich expression

$$\widehat{\mathrm{Cov}}(\hat{\theta}) = J_\delta^{-1} J_0 J_\delta^{-1},$$

as described in McInerney and Burke (2026). Here, "sandwich" refers to the curvature-based adjustment for penalisation in this expression. The middle matrix J_0 is the unpenalised observed-information approximation rather than an empirical outer product of observation-level scores, so the result is not a heteroskedasticity-robust covariance estimator. The calculation treats the observed design as fixed and uses independent homoskedastic Gaussian errors for continuous responses or independent Bernoulli responses for binary fits. When `decay = 0`, this reduces to the inverse observed-information approximation. The package checks the numerical behaviour of J_δ before computing the inverse.

The fitted parameter vector and covariance matrix are used to construct the Wald-type summaries and PCE plots described below. These quantities are conditional on the fitted architecture, value of `decay`, and local solution. They are not post-selection summaries and do not account for uncertainty arising from choosing the architecture, penalty, or initialisation. Crucially, numerical invertibility of J_δ establishes only that the covariance approximation can be computed; it does not establish that Wald reference distributions or delta-method coverage are well calibrated. The p-values and intervals reported by **statnnet** are therefore approximate and are empirically supported only under the simulation regimes studied in McInerney and Burke (2026).

2.1 Model selection

For continuous responses, `select_nnet()` implements the procedure developed in McInerney and Burke (2025). Candidate models are unpenalised single-hidden-layer Gaussian networks, and each candidate is fitted from several random initialisations. For a candidate with K network weights, let $\hat{\theta}$ denote the best converged fit found across the specified initialisations. The Gaussian log-likelihood evaluated at this retained fit includes the estimate $\hat{\sigma}^2 = n^{-1} \sum_i \{y_i - \mathrm{NN}(x_i, \hat{\theta})\}^2$, and the default comparison criterion is

$$\mathrm{BIC} = -2\ell(\hat{\theta}, \hat{\sigma}^2) + \log(n)(K + 1),$$

where the additional parameter is the residual variance. The search first compares hidden-layer sizes, then removes input terms while BIC improves, and finally alternates local hidden-layer and input changes to fine-tune the selected architecture. Only converged candidate fits are eligible for selection,

and the returned `"statnnet_selection"` object records the candidate path and start-level convergence diagnostics. Because the fitting objective is non-convex, the retained fit is the best solution found rather than a guaranteed global likelihood maximum. The alternating search is likewise path-dependent and does not exhaustively establish the global minimum BIC over all possible architectures. The package therefore uses BIC as the comparison criterion defined by the published selection procedure; its usual regular-model marginal-likelihood interpretation is not automatic for neural networks with equivalent or redundant parameterisations. For formula inputs, all model-matrix columns generated by the same term are selected together; for matrix inputs, each column is a selectable input. The current formula interface supports main-effect terms only: interactions are rejected rather than searched or partially removed.

The selection procedure is currently restricted to Gaussian responses and `decay = 0`, matching the published method. Binary-response and penalised models may still be supplied directly to `statnnet()` for conditional interpretation, but they are not currently selected by `select_nnet()`.

2.2 Wald-type summaries

One aim of **statnnet** is to provide regression-style summaries for supported `"nnet"` fits. Given an estimate $\hat{\theta}$ and an estimated covariance matrix $\widehat{\mathrm{Cov}}(\hat{\theta})$, the package computes Wald-type summaries for both individual weights and groups of weights.

For an individual parameter θ_m, the null hypothesis

$$H_0 : \theta_m = 0$$

is summarised using the statistic

$$W_m = \frac{\hat{\theta}_m^2}{\widehat{\mathrm{Cov}}(\hat{\theta})_{mm}},$$

which is compared to a χ_1^2 reference distribution. This produces a parameter-level summary similar in form to the output from classical regression models. However, individual neural-network weights are usually difficult to interpret in isolation, because the effect of an input variable is distributed across multiple connections and depends on the rest of the fitted network.

For this reason, **statnnet** also provides covariate-level Wald-type summaries. For a single-hidden-layer FNN, the contribution of covariate x_j to the hidden layer is represented by the vector

$$\omega_j = (\omega_{j1}, \ldots, \omega_{jq})^T,$$

which contains all weights connecting covariate j to the hidden nodes. The package therefore computes a multiple-parameter Wald-type statistic for

$$H_0 : \omega_j = 0_q.$$

If S_j is the matrix selecting the elements of θ corresponding to ω_j, then the relevant covariance matrix is

$$\widehat{\Sigma}_{\omega_j} = S_j \widehat{\mathrm{Cov}}(\hat{\theta}) S_j^T.$$

The corresponding statistic is

$$W_j = \hat{\omega}_j^T \widehat{\Sigma}_{\omega_j}^{-1} \hat{\omega}_j.$$

Define the shrinkage matrix $A = J_\delta^{-1} J_0$. The statistic is compared to a chi-squared reference distribution with effective degrees of freedom

$$\tilde{q}_j = \mathrm{tr}(S_j A S_j^T),$$

which is approximately q when `decay` is small. It provides a parameter-based input-level summary. The null is specifically that all input-to-hidden weights for the covariate are zero. This condition is sufficient to exclude dependence through that input, but it is not necessary for functional invariance: non-zero input weights can have little or no effect on the fitted response when outgoing weights are zero, hidden units are saturated, or contributions cancel. The grouped Wald summary should therefore be interpreted together with the function-scale effect summaries below.

For categorical covariates represented by several dummy variables, **statnnet** extends this idea by testing all associated input nodes jointly. For example, if a categorical covariate with $l + 1$ levels is represented using l dummy variables, and the network has q hidden nodes, the corresponding grouped test involves lq input-to-hidden weights. Using the corresponding $lq \times K$ selection matrix in the expression above gives effective degrees of freedom that are approximately lq when `decay` is small. This gives an analysis-of-variance-style summary for categorical predictors, allowing the user to assess the contribution of the original covariate rather than each dummy variable separately. The `anova()`

method reports separate grouped Wald tests within the retained fit. It does not perform sequential analysis of variance, likelihood-ratio testing, or model comparison, and its term-wise p-values are not adjusted for multiple testing.

2.3 Covariate-effect estimands

In addition to assessing whether a covariate contributes to the fitted model, it is useful to describe the nature of that contribution. For this purpose, **statnnet** provides two related but distinct finite-difference estimands. The partial-dependence PCE is used for effect curves and can also be averaged to a scalar contrast; the default `summary()` instead reports a faster observed-row predictive finite difference.

Partial-dependence PCE curves

Partial dependence plots describe how the average fitted response changes as one covariate is varied while the remaining covariates are held at their observed values. For covariate j, the partial dependence function is estimated as

$$\overline{\mathrm{NN}}(x^{(j)}) = \frac{1}{n}\sum_{i=1}^{n} \mathrm{NN}(x_i^{(j,x^{(j)})}, \hat{\theta}),$$

where $x_i^{(j,a)}$ denotes the covariate vector for observation i with coordinate j set to a; hence, $x_i^{(j,x^{(j)})}$ has the jth covariate set to the displayed value $x^{(j)}$.

The PCE is a finite-difference version of this quantity. For a d-unit increase in covariate j, the PCE is defined as

$$\hat{\beta}(x^{(j)}, d) = \overline{\mathrm{NN}}(x^{(j)} + d) - \overline{\mathrm{NN}}(x^{(j)}).$$

This quantity has an interpretation close to a regression coefficient: it estimates the average change in the fitted response associated with a d-unit increase in a covariate. The value of d can be chosen by the user; common choices are $d = 1$ or d equal to one empirical standard deviation of the covariate.

For binary covariates, the PCE corresponds to the average change in the fitted response when the covariate changes from 0 to 1. For continuous covariates, plotting $\hat{\beta}(x^{(j)}, d)$ over a sequence of values shows how the covariate effect varies across the covariate range. When the PCE is approximately constant, the partial-dependence function is approximately linear over the displayed covariate range; substantial variation indicates a non-linear average effect on that scale.

Uncertainty intervals for the PCE can be computed using the approximate sampling distribution of $\hat{\theta}$. For the delta method, the gradient of the PCE with respect to $\hat{\theta}$ is used:

$$\nabla_{\hat{\theta}}\hat{\beta}(x^{(j)}, d) = \frac{1}{n}\sum_{i=1}^{n}\left[\nabla_{\hat{\theta}}\mathrm{NN}(x_i^{(j,x^{(j)}+d)}, \hat{\theta}) - \nabla_{\hat{\theta}}\mathrm{NN}(x_i^{(j,x^{(j)})}, \hat{\theta})\right].$$

Hence,

$$\widehat{\mathrm{Var}}\left(\hat{\beta}(x^{(j)}, d)\right) \approx \nabla_{\hat{\theta}}\hat{\beta}(x^{(j)}, d)^T \widehat{\mathrm{Cov}}(\hat{\theta}) \nabla_{\hat{\theta}}\hat{\beta}(x^{(j)}, d).$$

This gives an analytic approximation to the variance, and, therefore, the standard error of the PCE at each plotted covariate value. The corresponding normal-approximation intervals are pointwise intervals, not simultaneous confidence bands over the complete curve. Alternatively, pointwise percentile intervals can be obtained by simulating parameter vectors from the approximate sampling distribution of $\hat{\theta}$ and recomputing the PCE for each simulated set of parameters as in Mandel (2013). Both forms of interval are conditional on the selected architecture, value of `decay`, local solution, approximate parameter covariance, empirical covariate rows, finite-difference step, and any centring or scaling constants. They propagate uncertainty in $\hat{\theta}$ while treating the observed design and preprocessing quantities as fixed; they therefore do not include sampling variation in the empirical covariate distribution or define full intervals for a population-average effect.

Scalar partial-dependence and observed-row effects

The partial-dependence PCE curve can be reduced to a single point-estimate effect by averaging it over the observed values of the focal covariate:

$$\hat{\tau}_j^{\mathrm{PD}} = \frac{1}{n}\sum_{i=1}^{n}\hat{\beta}(x_{ij}, d).$$

This average partial-dependence contrast is available from `summary(object,effects = "partial")` and is accompanied by a standard error. Because its direct calculation averages over all combinations

of the observed focal and remaining covariate values, its computational cost can be appreciable.

For continuous covariates, `summary(object)` defaults to `effects = "rowwise"` and reports the faster average observed-row predictive finite difference

$$\hat{\tau}_j^{\text{row}} = \frac{1}{n}\sum_{i=1}^{n}\left\{\text{NN}(x_i + de_j, \hat{\theta}) - \text{NN}(x_i, \hat{\theta})\right\},$$

where e_j selects covariate j. This estimand changes the focal covariate within each observed row, averages the resulting prediction differences, and requires only $O(n)$ prediction evaluations. It generally differs from the average partial-dependence contrast when the fitted network contains interactions; the two coincide for additive fitted response functions. For binary and categorical covariates, the fixed reference contrasts have the same rowwise and partial-dependence interpretations.

The interface and terminology follow this distinction: `summary(object)` returns $\hat{\tau}_j^{\text{row}}$, `summary(object,effects = "partial")` returns $\hat{\tau}_j^{\text{PD}}$, and `pce()` and `plot()` return the partial-dependence curve $\hat{\beta}(x^{(j)}, d)$. Neither scalar estimand is labelled as the other in the printed output. Together, the Wald-type summaries and these effect estimands provide the two pieces of information usually sought from regression-style output: whether a covariate appears to contribute to the fitted model, and how the fitted response changes with that covariate.

3 Relationship to existing software

The role of **statnnet** is most clearly understood in relation to **nnet** itself. The `nnet()` function fits single-hidden-layer neural networks using the BFGS algorithm and provides methods for prediction, together with access to the fitted weights, residuals, objective value, convergence code, and Hessian. These quantities provide the computational basis for further statistical summaries, but **nnet** does not itself provide covariate-level Wald-type summaries or PCE plots with uncertainty estimates. The purpose of **statnnet** is to supply model-selection and interpretation layers for supported `"nnet"` models.

This close dependence on **nnet** is a deliberate design choice. The package does not attempt to translate objects from several neural-network frameworks into a common representation, and it does not claim that the same covariance calculations are appropriate for arbitrary architectures or fitting objectives. Instead, the supported parameterisation, activation functions, weight ordering, objective, and convergence information are tied directly to those used by **nnet**. This narrower interface makes it possible to validate the required model information and to issue warnings when a fitted network lies outside the intended statistical setting.

Model-agnostic interpretability packages provide an important first comparison. **pdp** computes partial-dependence and individual-conditional-expectation curves for many fitted model classes (Greenwell, 2017), while **iml** (Molnar et al., 2018), **DALEX** (Biecek, 2018), and the archived **vip** package (Greenwell and Boehmke, 2020) provide broader collections of partial dependence, accumulated local effects, variable importance, surrogate models, or Shapley-value explanations. These tools are valuable because they compare prediction functions across many model classes, but they are not designed around the likelihood, weight ordering, and curvature of a particular `"nnet"` fit.

Two packages are closer in their explicit support for neural-network objects. **NeuralNetTools** provides methods for `"nnet"` and other neural-network classes, including network diagrams, connection-weight variable-importance measures, and Lek profile sensitivity plots (Beck, 2018). **NeuralSens** calculates derivatives of network outputs with respect to inputs and summarises their distribution across the training observations; it supports `"nnet"` and several other multilayer-perceptron implementations (Pizarroso et al., 2022). These packages provide useful complementary descriptions of network structure, importance, and local sensitivity. Their primary targets differ from the finite-difference partial-dependence and observed-row estimands in **statnnet**, and they do not reconstruct the **nnet** likelihood curvature to obtain the grouped parameter Wald summaries used here.

cito is the closest comparison in terms of regression-like effect output. It fits separate **torch**-based neural networks and reports permutation importance, average conditional effects, partial-dependence and accumulated-local-effect plots; bootstrap refits can provide intervals and p-values (Amesöder et al., 2024). Thus, **cito** has broader architecture and fitting ambitions, whereas **statnnet** is restricted to `nnet`, provides information-criterion selection for its supported Gaussian setting, and uses an analytic covariance approximation for interpretation without bootstrap refitting. The approaches are complementary rather than interchangeable because they operate on different fitted model classes and use different inferential mechanisms.

Table 1 summarises the closest comparisons.

Table 1: Closest R packages for neural-network interpretation and effect summaries.

Package	Fit or interface	Primary output
statnnet	Selects or accepts an 'nnet' fit.	IC selection; row/PD effects; grouped Wald.
pdp	Many model classes.	PD and ICE curves.
NeuralNetTools	'nnet' and other NNs.	Plots, weight importance, Lek profiles.
NeuralSens	'nnet' and other MLPs.	Input-derivative sensitivities.
cito	Own 'torch' fits.	ACE, PDP/ALE, importance; bootstrap.

Unlike **statnnet**, the comparison packages do not combine information-criterion selection of an `"nnet"` architecture with diagnostics and conditional regression-style summaries for the retained fit. **cito** fits a separate model, while the other packages describe different aspects of an existing prediction function. The approaches are therefore complementary rather than numerically interchangeable.

4 Using statnnet

We demonstrate the complete **statnnet** workflow using the Auto MPG data from the UCI Machine Learning Repository (Quinlan, 1993). This example makes model selection the entry point to the analysis: `select_nnet()` chooses the input terms and hidden-layer size, the selected fit is checked before covariance-based interpretation, and `statnnet()` supplies regression-style summaries and covariate-effect plots. The information criterion selects an unpenalised model, whereas a subsequent, explicitly reported sensitivity analysis is used to obtain a better-conditioned fit of the selected structure. The results below were generated with **statnnet** version 0.1.0; the reproduction bundle records the complete R session information.

4.1 Data and preprocessing

The source data contain fuel economy and vehicle characteristics for 398 cars. Following the dataset documentation, six observations with missing horsepower are omitted, leaving 392 complete cases. The car-name identifier is not retained. The data are distributed by UCI under the Creative Commons Attribution 4.0 International licence.

```
library(statnnet)
library(nnet)

data("auto_mpg", package = "statnnet")

str(auto_mpg)

#> 'data.frame':    392 obs. of  8 variables:
#>  $ mpg         : num  18 15 18 16 17 15 14 14 14 15 ...
#>  $ cylinders   : Factor w/ 5 levels "3","4","5","6",..: 5 5 5 5 5 5 5 5 5 5 ...
#>  $ displacement: num  307 350 318 304 302 429 454 440 455 390 ...
#>  $ horsepower  : int  130 165 150 150 140 198 220 215 225 190 ...
#>  $ weight      : int  3504 3693 3436 3433 3449 4341 4354 4312 4425 3850 ...
#>  $ acceleration: num  12 11.5 11 12 10.5 10 9 8.5 10 8.5 ...
#>  $ model_year  : int  70 70 70 70 70 70 70 70 70 70 ...
#>  $ origin      : Factor w/ 3 levels "USA","Europe",..: 1 1 1 1 1 1 1 1 1 1 ...
```

Cylinder count and geographic origin are factors, so `select_nnet()` removes or retains all model-matrix columns associated with either term as a group. The continuous predictors and response are standardised before fitting. Reported effects and predictions are converted back to miles per gallon where appropriate.

```
continuous_predictors <- c(
  "displacement", "horsepower", "weight",
  "acceleration", "model_year"
)
predictor_center <- vapply(
  auto_mpg[continuous_predictors], mean, numeric(1)
```

```
)
predictor_scale <- vapply(
  auto_mpg[continuous_predictors], sd, numeric(1)
)
response_center <- mean(auto_mpg$mpg)
response_scale <- sd(auto_mpg$mpg)

auto_mpg_analysis <- auto_mpg
auto_mpg_analysis[continuous_predictors] <- Map(
  function(x, center, scale) (x - center) / scale,
  auto_mpg[continuous_predictors],
  predictor_center,
  predictor_scale
)
auto_mpg_analysis$mpg <- (
  auto_mpg$mpg - response_center
) / response_scale

auto_mpg_formula <- mpg ~ cylinders + displacement + horsepower +
  weight + acceleration + model_year + origin
```

4.2 Selecting the network

The search compares hidden-layer sizes, removes formula terms while BIC improves, and then fine-tunes the input and hidden layers. Each candidate is fitted from 20 pseudo-random initialisations generated under a fixed seed.

```
selection <- select_nnet(
  auto_mpg_formula,
  data = auto_mpg_analysis,
  max_size = 4,
  n_starts = 20,
  criterion = "BIC",
  maxit = 3000,
  seed = 20260920,
  progress = FALSE
)

selection
```

```
#> statnnet model selection
#> Call:
#> select_nnet(x = auto_mpg_formula, data = auto_mpg_analysis, max_size = 4,
#>     n_starts = 20, criterion = "BIC", maxit = 3000, seed = 20260920,
#>     progress = FALSE)
#> Selected model: 5 input columns, 2 hidden units, 15 network weights
#> BIC: 309.698; 25 candidate models; 500 converged starts
#> Selected terms: displacement, horsepower, weight, acceleration, model_year
#> Excluded terms: cylinders, origin
```

The selected model has two hidden units and retains displacement, horsepower, weight, acceleration, and model year. Cylinder count and origin are removed as complete factor terms, and all 500 candidate starts report convergence code zero. The evaluated candidate path is retained rather than reducing selection to only the final model. Across repeated renders, the serial search took approximately 10–15 seconds on the Windows x86-64 session recorded in the reproduction bundle; this timing is hardware-specific. For a fixed candidate path, computational work grows approximately linearly with `n_starts`, while increasing `max_size` raises both the number of fitted weights and the cost of each optimisation; additional terms can also lengthen the data-dependent search path. For longer searches, `progress = TRUE` prints candidate-level progress, and the returned path and start-level diagnostics should be retained rather than increasing `n_starts` without inspection.

```
path <- selection_path(selection)
```

```
path[path$selected, c(
  "phase", "candidate", "included_terms",
  "size", "criterion_value", "converged_starts"
)]

#>    phase        candidate
#> 12 input remove_cylinders
#>                                                 included_terms size
#> 12 displacement + horsepower + weight + acceleration + model_year    2
#>    criterion_value converged_starts
#> 12        309.6982              20
```

The selected architecture and terms are conditional on the candidate sizes, criterion, start distribution, number of starts, and seed. The path and start-level diagnostics should therefore be retained when reporting the analysis.

4.3 Checking and refitting the selected structure

Calling `statnnet()` on the selection result applies the same validation used for an independently fitted `"nnet"` object. The selected unpenalised network has an available covariance matrix but triggers the conditioning warning. Its reciprocal information condition number is approximately 1.75×10^{-10}, so convergence alone is not sufficient justification for interpreting covariance-based summaries.

```
selected_model <- suppressWarnings(
  statnnet(selection, objective_tol = 1e-4)
)

selected_model

#> statnnet: statistical summary of a fitted nnet model
#> Formula: mpg ~ displacement + horsepower + weight + acceleration + model_year
#> Architecture: 5-2-1; 15 weights; n = 392
#> Response: continuous; decay: 0
#> Selection: BIC = 309.698; uncertainty is conditional on the selected model
#> nnet convergence code: 0
#> Covariance: available (reciprocal condition number 1.75e-10)
```

The selected terms and hidden-layer size are held fixed while the same structure is refitted over a short decay grid. This is a diagnostic sensitivity analysis, not an extension of the BIC selection procedure. At each positive decay value, the selected weights and 19 pseudo-random starts generated under a fixed seed are considered, and the converged fit with the smallest penalised objective is retained.

Table 2: Conditioning sensitivity for the selected two-hidden-unit Auto MPG architecture.

Decay	Training RMSE (MPG)	Reciprocal condition number	Conditioning warning
0.000	2.482	1.75e-10	TRUE
0.001	2.544	7.49e-07	TRUE
0.010	2.588	8.28e-06	TRUE
0.020	2.611	1.29e-05	FALSE
0.050	2.684	6.54e-05	FALSE
0.100	2.688	8.74e-05	FALSE

A decay of 0.02 is the first value in this grid that removes the package's conditioning warning. It increases training RMSE modestly relative to the unpenalised selected fit. The value is not described as selected by BIC; it is an explicitly reported stabilising refit used for the conditional summaries below.

```
chosen_index <- which(!decay_summary$conditioning_warning)[1]
regularized_fit <- decay_fits[[chosen_index]]$fit
snn <- decay_fits[[chosen_index]]$model

snn
```

```
#> statnnet: statistical summary of a fitted nnet model
#> Formula: mpg ~ displacement + horsepower + weight + acceleration + model_year
#> Architecture: 5-2-1; 15 weights; n = 392
#> Response: continuous; decay: 0.02
#> nnet convergence code: 0
#> Covariance: available (reciprocal condition number 1.29e-05)
```

4.4 Interpreting the selected model

The default summary reports average observed-row predictive finite differences and grouped Wald-type summaries. For a one-standard-deviation change in each retained predictor, the following table converts the response changes, standard errors, and pointwise confidence intervals back to miles per gallon.

Table 3: Average fitted MPG changes for one-standard-deviation increases in the retained predictors, with conditional pointwise 95% delta-method confidence intervals.

Predictor	Estimate (MPG)	SE (MPG)	95% lower	95% upper
displacement	0.231	0.538	-0.824	1.286
horsepower	-1.960	0.512	-2.964	-0.957
weight	-4.136	0.432	-4.982	-3.290
acceleration	-0.173	0.212	-0.589	0.242
model_year	3.365	0.237	2.901	3.829

These are summaries of the fitted response function, not causal effects. In particular, displacement, horsepower, and weight are strongly correlated, so their conditional descriptions should not be read as effects of interventions. The grouped summaries use the familiar `anova()` interface:

```
anova(snn)
```

```
#> Grouped input-to-hidden Wald summaries
#>          term model_columns n_weights effective_df wald_chisq      p_value
#>  displacement  displacement         2     1.918217    8.21192 1.504136e-02
#>    horsepower    horsepower         2     1.798030   27.28933 8.461245e-07
#>        weight        weight         2     1.934151   31.80612 1.107483e-07
#>  acceleration  acceleration         2     1.979633   13.17707 1.340039e-03
#>    model_year    model_year         2     1.955676   72.47082 1.669613e-16
```

The reference distributions remain conditional on the selected structure, decay value, and retained local optimum. They do not account for model-selection uncertainty.

The fitted architecture is displayed using `plot_network()`.

```
network_par <- par(cex = 1.18)
plot_network(snn, annotate = "none")


par(network_par)
```

A PCE curve describes how a finite change in one predictor alters the average fitted response while the remaining predictors are averaged over their observed rows. For vehicle weight, the default step is one observed standard deviation on the fitted scale.

```
weight_pce <- pce(
  snn, variable = "weight", uncertainty = "delta"
)
weight_pce$estimate_mpg <- response_scale * weight_pce$estimate
weight_pce$conf_low_mpg <- response_scale * weight_pce$conf_low
weight_pce$conf_high_mpg <- response_scale * weight_pce$conf_high

plot(
  weight_pce$value,
```

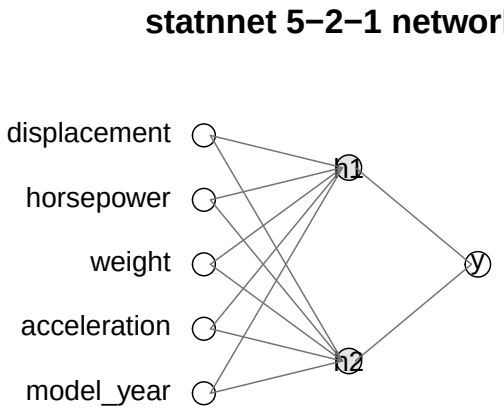


Figure 1: Regularised refit of the selected Auto MPG architecture. The fitted network has five retained inputs, two hidden units, and one linear output; edge annotations are suppressed to emphasise the selected structure.

```
  weight_pce$estimate_mpg,
  type = "l",
  ylim = range(weight_pce[c("conf_low_mpg", "conf_high_mpg")]),
  xlab = "Standardised vehicle weight",
  ylab = "Partial covariate effect (MPG)"
)
lines(weight_pce$value, weight_pce$conf_low_mpg, lty = 2)
lines(weight_pce$value, weight_pce$conf_high_mpg, lty = 2)
```

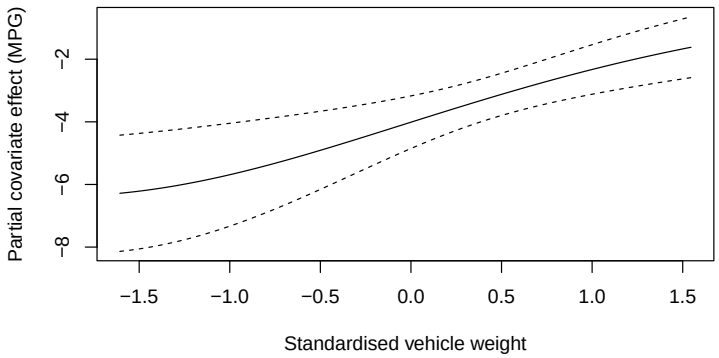


Figure 2: Partial covariate-effect curve for standardised vehicle weight in the selected and regularised Auto MPG model. The horizontal axis is in standard deviations of vehicle weight and the vertical axis is in MPG. Dashed lines are pointwise delta-method intervals conditional on the selected structure, decay value, and local optimum.

The retained fit also supports prediction. Because the response was standardised before fitting, predictions are converted back to MPG.

```
new_auto_mpg <- auto_mpg_analysis[
```

```
  1:5,
  setdiff(names(auto_mpg_analysis), "mpg")
]
predicted_mpg <- response_center +
  response_scale * predict(snn, newdata = new_auto_mpg)

data.frame(
  observed_mpg = auto_mpg$mpg[1:5],
  predicted_mpg = as.numeric(predicted_mpg)
)

#>   observed_mpg predicted_mpg
#> 1           18      15.52710
#> 2           15      14.47749
#> 3           18      15.41556
#> 4           16      15.36053
#> 5           17      15.67742
```

This workflow keeps the package roles explicit: `select_nnet()` selects an unpenalised Gaussian architecture and terms, diagnostics determine whether the covariance calculation is numerically available, and `statnnet()` supplies conditional summaries for the retained fit. These diagnostics do not determine whether the resulting inferential quantities are calibrated.

4.5 Binary responses and grouped factors

Binary-response models are augmented directly rather than selected by `select_nnet()`. The following compact example creates a two-level response for illustration, retains the three-level `origin` factor in the fitted model, and then tests its two model-matrix columns jointly.

```
binary_data <- auto_mpg
binary_data$weight <- as.numeric(scale(binary_data$weight))
binary_data$mpg_high <- factor(
  ifelse(
    binary_data$mpg > median(binary_data$mpg),
    "high", "low"
  ),
  levels = c("low", "high")
)

set.seed(20260921)
binary_fit <- nnet(
  mpg_high ~ weight + origin,
  data = binary_data,
  size = 1,
  decay = 0.05,
  Hess = TRUE,
  maxit = 3000,
  trace = FALSE
)
binary_model <- statnnet(
  binary_fit,
  data = binary_data,
  response = "binary"
)

anova(binary_model)

#> Grouped input-to-hidden Wald summaries
#>    term             model_columns n_weights effective_df wald_chisq
#>  weight                    weight         1    0.9699942   17.37182
#>  origin originEurope, originJapan         2    1.9803351    1.28961
#>       p_value
#>  2.883086e-05
#>  5.199502e-01
```

The `origin` row is therefore a grouped two-degree-of-freedom summary rather than two unrelated dummy-variable tests. The simulation evidence cited in this paper concerns continuous Gaussian responses; no binary-response calibration study is supplied here. Accordingly, the binary Wald p-values and intervals should be treated as exploratory. As elsewhere, the reference distribution is conditional on the fitted architecture, decay value, and retained local optimum, and the constructed response is solely a software illustration.

5 Supported objects and methods

This section records the supported-model contract and user-facing methods; the corresponding workflow was demonstrated in Section 2.4.

5.1 Supported model contract

statnnet supports single-hidden-layer `"nnet"` models fitted to tabular data. Continuous responses require a linear output and are treated as Gaussian non-linear regression models; binary responses require a single logistic output with entropy fitting and are treated as Bernoulli regression models. Skip-layer connections, softmax or censored outputs, multiple output nodes, and objects fitted by other neural-network packages are outside this scope. Here, support means that the package can validate the fitted object and compute the documented quantities; it is not a guarantee of inferential calibration.

Inputs may be continuous, binary, or categorical. When the original formula and data are supplied, the mapping from the model-matrix columns to the original covariates is retained so that all dummy variables associated with a factor can be summarised jointly. The original `"nnet"` fit is also retained, allowing prediction to be delegated to `predict.nnet()`. The `select_nnet()` search is restricted to main-effect formulas and does not support interaction terms; a supported interaction model fitted directly by `nnet()` may still be augmented conditionally by `statnnet()`. For covariance-based interpretation, the empirical results in McInerney and Burke (2026) favour parsimonious architectures and motivate keeping the hidden layer no larger than the fitted input layer. The package records the hidden-to-input ratio and warns when the number of hidden nodes exceeds the number of fitted input nodes. Satisfying this rule removes one identified risk but does not by itself establish calibration.

5.2 Requirements and diagnostic checks

The input to `statnnet()` must inherit from class `"nnet"`, and the original training data must always be supplied. The formula may be omitted only when it can be recovered from an `"nnet.formula"` object. The constructor checks the architecture, response type, convergence code, and agreement between the reconstructed model matrix and the fitted network.

Before computing covariance-based summaries, the package checks the **nnet** convergence code and the availability and numerical behaviour of the Hessian. If the penalised curvature is effectively singular or cannot be inverted reliably, the affected covariance-based summaries are not reported. These checks prevent clearly unsuitable numerical calculations from being reported without notice, but they do not test the accuracy of the chi-squared or normal reference approximations. For binary responses, the diagnostics additionally note that frequentist calibration has not been established by the package authors.

5.3 User-facing functions and methods

The main user-facing functions and methods are summarised in Table 4. Together, these functions select or augment a fitted `"nnet"` model while preserving the established optimisation and prediction engine.

Table 4: Main user-facing functions and methods in statnnet.

Function	Purpose
select_nnet()	Selects input terms and hidden-layer size.
selection_path()	Returns the evaluated candidate-model path.
statnnet()	Validates and augments an `nnet` fit or selection.
print()	Prints architecture and diagnostics.
summary()	Reports effects and grouped Wald summaries.
anova()	Reports grouped Wald summaries.
coef()	Returns named fitted network weights.
vcov()	Returns the covariance estimate.
pce()	Computes finite-difference covariate effects.
plot()	Plots PCEs and pointwise intervals.
plot_network()	Draws an annotated network.
predict()	Delegates prediction to the retained fit.

nnet therefore remains responsible for optimisation and prediction, while `select_nnet()`, `statnnet()`, and their methods provide model selection, statistical summaries, diagnostics, and visualisations.

6 Practical guidance and limitations

The covariance-based summaries are intended for small, parsimonious networks whose fitted parameters are locally well identified. Increasing the hidden-layer size can improve flexibility, but it also increases the number of weights and the likelihood of redundant hidden nodes or near-singular curvature. More importantly, the continuous-response simulations in McInerney and Burke (2026) show that Wald tests can be severely anti-conservative for over-sized architectures even when penalisation makes covariance estimation numerically feasible. Those results motivate keeping the hidden layer no larger than the fitted input layer and using a modest positive `decay`; the package emits an architecture warning when the first recommendation is violated. The `decay` value is incorporated into the covariance calculation exactly as described in Section 2.2; both the hidden-layer size and `decay` should therefore be reported and examined in sensitivity analyses.

Convergence of `nnet()` and a numerically invertible information matrix are necessary computational checks, not sufficient evidence of calibrated inference. Wald p-values and delta-method intervals are approximate and should be interpreted only in relation to the data-generating settings, sample sizes, architectures, and penalties studied empirically. The calibration studies cited here concern continuous Gaussian responses, so binary-response p-values and intervals are exploratory unless supported by a separate validation study for the application at hand.

The fitting objective is non-convex, so different initial values can lead to different local solutions. Using several initialisations reduces the risk of reporting a poor solution, but does not account for the uncertainty introduced by selecting among them. Similarly, the reported reference distributions are conditional on the chosen architecture and `decay` value. If these are selected using the same data, the output should not be interpreted as post-selection inference; the selection procedure and the stability of the resulting summaries should be reported separately.

Individual neural-network weights should be interpreted cautiously. A single input-to-hidden-layer weight is only one component of the fitted function, and its meaning depends on the remaining weights in the network. In addition, hidden nodes can be permuted without changing the fitted input-output function, and sign or label symmetries can lead to equivalent parameterisations. For this reason, **statnnet** reports individual-weight summaries mainly as diagnostic quantities. In most applications, input-level summaries, covariate-level summaries, and PCE plots will be more useful for interpretation.

Categorical covariates should generally be interpreted at the covariate level rather than only at the dummy-variable level. When a factor is represented by several dummy variables, separate input-level summaries can be difficult to interpret in isolation. The `anova()` method therefore groups the associated input nodes and provides a covariate-level Wald-type summary. This gives an output closer in spirit to an analysis-of-variance table for a standard regression model.

Both covariate-effect estimands should be interpreted as summaries of the fitted model, not as causal effects. The partial-dependence PCE averages predictions over the empirical distribution of the remaining covariates at each focal value; the default observed-row effect instead changes the focal value within each observed row before averaging the prediction differences. They can differ materially under interactions, and neither resolves confounding or a lack of support caused by

correlated predictors. As with partial dependence plots more generally, interpretation can be more difficult when covariates are strongly correlated or when the plot evaluates combinations of covariate values that are rare in the observed data. For default continuous PCE curves, **statnnet** keeps both finite-difference endpoints within the observed univariate covariate range. User-supplied evaluation values and average effects are not restricted in this way and can still involve extrapolation, particularly near a boundary or with a large finite-difference step. The plots are therefore best viewed as descriptive tools for understanding the fitted prediction function.

The inferential summaries produced by **statnnet** do not replace model checking or predictive validation. Users should still assess predictive performance using appropriate validation procedures, examine the sensitivity of the fitted model to architecture and penalty choices, and consider whether the chosen neural network is appropriate for the scientific or applied question. The package makes fitted shallow FNNs more interpretable, but the quality of the resulting interpretation depends on the quality and stability of the fitted model.

7 Conclusion

The **statnnet** package provides statistical selection and interpretation for supported single-hidden-layer neural networks fitted using **nnet**. Rather than introducing a new optimiser or a common interface for several neural-network frameworks, the package coordinates `nnet()` candidate fits during information-criterion selection and augments a selected or independently fitted `"nnet"` object with the information required for regression-style summaries, diagnostic checks, and covariate-effect visualisations. In this way, **statnnet** makes a deliberately restricted class of shallow neural networks behave more like standard statistical model objects in R.

The package implements information-criterion selection of input terms and hidden-layer size, regression-style summaries, Wald-type summaries for individual weights and groups of weights, analysis-of-variance-style summaries for covariates, average observed-row predictive finite differences, partial-dependence PCE plots and contrasts with uncertainty estimates, architecture diagrams annotated with statistical summaries, and prediction methods for new observations. These tools complement **nnet** by providing outputs that are more familiar to users of classical regression models.

The statistical methodology underlying selection is developed in McInerney and Burke (2025), and the methodology underlying the interpretability summaries is developed in McInerney and Burke (2026). The contribution of the present article is to describe the corresponding R implementation and demonstrate how the methodology can be used in practice through a coherent package workflow. By combining the flexibility of a parsimonious single-hidden-layer network with regression-style output, **statnnet** provides a practical interface for statisticians who wish to use supported `"nnet"` models as statistical modelling tools rather than purely predictive algorithms.

Future development will remain centred on the **nnet** setting, with possible extensions including alternative uncertainty estimators and additional diagnostics for assessing the stability of fitted models. Support for deep-learning frameworks or arbitrary neural-network architectures is not an objective of the package. More generally, **statnnet** provides a focused extension for integrating small `"nnet"` models more naturally into the statistical modelling workflow in R.

8 Acknowledgements

This publication has emanated from research conducted with the financial support of Science Foundation Ireland under Grant number 18/CRT/6049. The second author was supported by the Confirm Smart Manufacturing Centre (https://confirm.ie/) funded by Science Foundation Ireland (Grant Number: 16/RC/3918). For the purpose of Open Access, the authors have applied a CC BY public copyright licence to any Author Accepted Manuscript version arising from this submission.

References

C. Amesöder, F. Hartig, and M. Pichler. cito: An R package for training neural networks using torch. *Ecography*, 2024(6):e07143, 2024. doi: 10.1111/ecog.07143. [p6]

M. W. Beck. NeuralNetTools: Visualization and analysis tools for neural networks. *Journal of Statistical Software*, 85(11):1–20, 2018. doi: 10.18637/jss.v085.i11. [p6]

P. Biecek. DALEX: Explainers for complex predictive models in R. *Journal of Machine Learning Research*, 19(84):1–5, 2018. URL https://jmlr.org/papers/v19/18-416.html. [p6]

J. Doohan, L. Kook, and K. Burke. Comparison of generalised additive models and neural networks in applications: A systematic review. *Expert Systems with Applications*, 309:131082, 2026. doi: 10.1016/j.eswa.2025.131082. [p1]

B. M. Greenwell. pdp: An R package for constructing partial dependence plots. *The R Journal*, 9(1): 421–436, 2017. doi: 10.32614/RJ-2017-016. [p6]

B. M. Greenwell and B. C. Boehmke. Variable importance plots—an introduction to the vip package. *The R Journal*, 12(1):343–366, 2020. doi: 10.32614/RJ-2020-013. [p6]

Y. LeCun, Y. Bengio, and G. Hinton. Deep learning. *Nature*, 521(7553):436–444, 2015. doi: 10.1038/nature14539. [p1]

M. Mandel. Simulation-based confidence intervals for functions with complicated derivatives. *The American Statistician*, 67(2):76–81, 2013. doi: 10.1080/00031305.2013.783880. [p5]

A. McInerney and K. Burke. A statistical modelling approach to feedforward neural network model selection. *Statistical Modelling*, 25(4):323–342, 2025. doi: 10.1177/1471082X241258261. [p2, 3, 15]

A. McInerney and K. Burke. Investigating statistical inference and covariate effects in shallow neural networks. *arXiv preprint arXiv:2311.08139*, 2026. doi: 10.48550/arXiv.2311.08139. [p2, 3, 13, 14, 15]

C. Molnar, B. Bischl, and G. Casalicchio. iml: An R package for interpretable machine learning. *Journal of Open Source Software*, 3(26):786, 2018. doi: 10.21105/joss.00786. [p6]

J. Pizarroso, J. Portela, and A. Muñoz. NeuralSens: Sensitivity analysis of neural networks. *Journal of Statistical Software*, 102(7):1–36, 2022. doi: 10.18637/jss.v102.i07. [p6]

R. Quinlan. Auto MPG. UCI Machine Learning Repository, 1993. Dataset, CC BY 4.0. [p7]

M. T. Ribeiro, S. Singh, and C. Guestrin. Why should i trust you?: Explaining the predictions of any classifier. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, KDD '16, pages 1135–1144, New York, NY, USA, 2016. Association for Computing Machinery. doi: 10.1145/2939672.2939778. [p1]

E. Strumbelj and I. Kononenko. An efficient explanation of individual classifications using game theory. *Journal of Machine Learning Research*, 11:1–18, 2010. URL https://www.jmlr.org/papers/v11/strumbelj10a.html. [p1]

W. N. Venables and B. D. Ripley. *Modern Applied Statistics with S*. Springer, New York, fourth edition, 2002. doi: 10.1007/978-0-387-21706-2. [p1]

Andrew McInerney
University of Limerick
Department of Mathematics and Statistics
Limerick, Ireland
ORCiD: 0000-0002-2348-2159
andrew.mcinerney@ul.ie

Kevin Burke
University of Limerick
Department of Mathematics and Statistics
Limerick, Ireland
kevin.burke@ul.ie